CHAPTER 04

A Framework For System Design Interviews

A 4-step playbook to structure any 45-60 minute design interview.

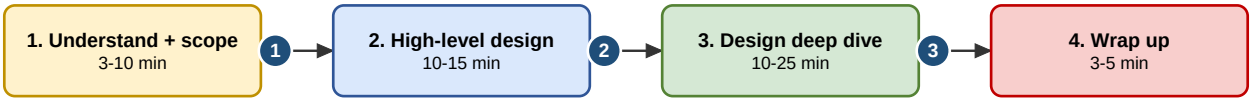
Jump to: What this chapter is about | The 4-step process | Deep dive | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. What this chapter is about

System design interviews are deliberately open-ended and ambiguous. The interviewer evaluates how you **collaborate, handle ambiguity, make trade-offs, and communicate** - not whether you reproduce a textbook architecture. This chapter gives a reusable 4-step process and a clear list of do's and don'ts.

Why interviewers ask it: A strong framework keeps you from rambling, running out of time, or diving into details before the big picture is agreed.

2. The 4-step process (time budget for a 45 min interview)



Step	What happens
1	Only move on once requirements, scale and assumptions are written on the board and the interviewer agrees. Never start drawing boxes before this.
2	Get **buy-in on the block diagram** (clients, APIs, services, stores) and walk 1-2 use cases through it. Then agree which 2-3 components to deepen.
3	Switch to wrap-up when time is nearly out: summarise, name the bottlenecks, failure modes and monitoring, and mention how it would evolve at 10x.

3. Deep dive

Step 1 - Understand the problem and establish design scope (3-10 min)

- Do **not** jump to a solution. Ask questions until requirements are clear; there is no right answer, only the best one for the stated constraints.
- Ask about: what features? how many users (DAU / MAU)? how fast will it grow in 3, 6, 12 months? which platforms (web / mobile)? what is the existing tech stack I can leverage? read:write ratio? latency, consistency, availability needs?
- Write down assumptions on the whiteboard; you will refer to them later.
- Example (news feed): mobile or web? what features (post, view feed)? sorted by time or ranking? how many friends? traffic volume? media support?

Step 2 - Propose a high-level design and get buy-in (10-15 min)

- Draw a **block diagram** with clients, APIs, services, DBs, caches, CDN, queues. Treat the interviewer as a teammate.
- Walk a couple of concrete use cases through the diagram; this surfaces missing pieces.
- Include API design and data schema if relevant. Do a back-of-the-envelope check to see whether the design fits the scale.
- Ask: 'Does this high-level direction look right before I go deeper?'

Step 3 - Design deep dive (10-25 min)

- Agree with the interviewer on which 2-3 components to dive into; priorities differ (e.g. for a chat system: delivery + sync, not login).
- Show depth on the **hard parts**: data model and storage choice, scaling the bottleneck, consistency, failure handling, hot spots.
- Always compare at least two options and say why you pick one; trade-off reasoning is the signal interviewers look for.
- Avoid unnecessary detail (e.g. exact ranking algorithm internals) unless asked.

Step 4 - Wrap up (3-5 min)

- Summarise the design; identify bottlenecks and possible improvements; do not claim 'it is perfect'.
- Discuss error cases (server failure, network loss), operational concerns (monitoring metrics, logs, deployment, rollback).
- Mention next scale curve: how would it change for 10x traffic? Extra features: privacy, GDPR, abuse, cost.

Do's

- Always ask for clarification; do not assume your assumption is correct.
- Understand the requirements before designing. Think out loud and keep communicating.
- Suggest multiple approaches if possible and explain pros/cons.
- Design the most critical components first. Bounce ideas off the interviewer.
- Never give up; if stuck, ask for hints.

Don'ts

- Do not come unprepared for typical interview questions.
- Do not jump into a solution without clarifying requirements and assumptions.
- Do not go into too much detail on a single component at the start; give the high-level design first.
- Do not hesitate to ask for hints if stuck. Do not think in silence.
- Do not think you are done after giving the design; get feedback early and often.

A reusable component checklist

- Clients -> DNS / CDN -> load balancer / API gateway -> stateless services -> cache -> DB (SQL / NoSQL) -> queue / stream -> workers -> object storage -> search index -> monitoring.
- For every box ask: scaling? failure? consistency? security? cost?

4. Common mistakes

- Spending 25 minutes on requirements, or zero minutes on them.
- Drawing a perfect final architecture with no justification.
- Neglecting to quantify; designing for Google scale when the brief is 10 K users (or the reverse).
- Forgetting to leave time for bottlenecks and failure modes.

5. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Let me start by asking a few questions so I design the right thing: which features matter most, how many users and how fast will that grow, which platforms, and what existing tech can I use? I'll write the assumptions on the board."
2. High-level design	12 min	"Here's a first block diagram with clients, load balancer, services and data stores. Let me walk through two core use cases to check it works, and do a quick estimate to see whether it fits the scale. Does this direction look right before I go deeper?"
3. Deep dive	22 min	"Which part would you like me to go deeper on? I'd suggest the one that is hardest to scale. I'll compare two options, state the trade-off, and pick one with a reason, then cover how it fails and recovers."
4. Wrap up	6 min	"To summarise the design: ... The main bottlenecks are ..., and I would monitor ... If traffic grew 10x, the first thing to change is ... I'd also want to cover privacy, abuse and cost if we had more time."

6. 60-second recap / practice checklist

- Scope -> Sketch -> Sink in (deep dive) -> Summarise.
- Ask, assume, quantify, draw, validate, deepen, trade-off, recap.
- Practice with a timer and a friend; record yourself and check the time split.

7. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• GeeksforGeeks: How to approach a system design problem
• Google SRE book: Service level objectives